

Write a program of Missionaries and Cannibals using python code

Aim :To solve the Missionaries and Cannibals problem using Python.

Procedure:

Represent the number of missionaries and cannibals on the left bank.

Move people across the river using a boat of capacity 2.

Ensure cannibals never outnumber missionaries on either bank.

Use Breadth-First Search (BFS) to find a valid sequence of moves.

Print the steps from the start state to the goal state.

Python Code:

```
from collections import deque

start = (3, 3, 1)    # (Missionaries, Cannibals, Boat)

goal = (0, 0, 0)

moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]

def valid(state):

    m, c, b = state

    if m < 0 or c < 0 or m > 3 or c > 3:

        return False

    if (m > 0 and m < c):

        return False

    if ((3-m) > 0 and (3-m) < (3-c)):

        return False

    return True

queue = deque([(start, [start])])
```

```
visited = set()

while queue:

    state, path = queue.popleft()

    if state == goal:

        print("Solution Path:")

        for step in path:

            print(step)

        break

    if state in visited:

        continue

    visited.add(state)

    m, c, b = state

    for dm, dc in moves:

        if b == 1:

            new = (m-dm, c-dc, 0)

        else:

            new = (m+dm, c+dc, 1)

        if valid(new):

            queue.append((new, path+[new]))
```

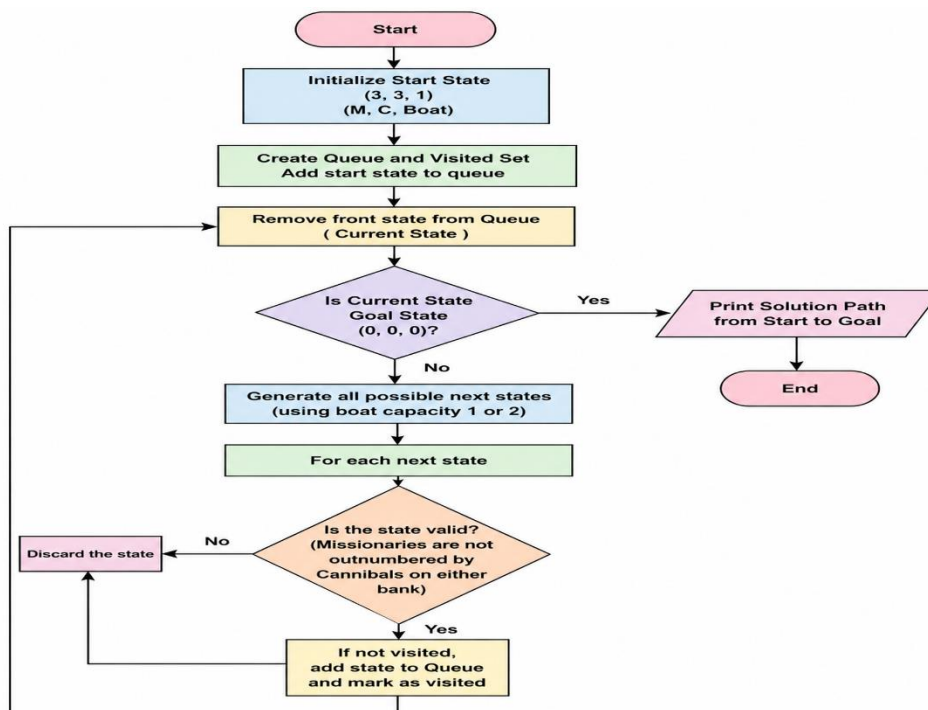
Output:

```
from collections import deque
start = (3, 3, 'L')
goal = (0, 0, 'R')
moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
def valid(m, c):
    return (0 <= m <= 3 and 0 <= c <= 3 and
            (m == 0 or m >= c) and
            (3-m == 0 or 3-m >= 3-c))
q = deque([(start, [start])])
visited = {start}
while q:
    (m, c, b), path = q.popleft()
    if (m, c, b) == goal:
        for state in path:
            print(state)
        break
    for dm, dc in moves:
        if b == 'L':
            nm, nc, nb = m-dm, c-dc, 'R'
        else:
            nm, nc, nb = m+dm, c+dc, 'L'
        if valid(nm, nc):
            state = (nm, nc, nb)
            if state not in visited:
```

```
(3, 3, 'L')
(3, 1, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 0, 'R')
```

```
=== Code Execution Success
```

Flowchart:



Result: Hence, Missionaries and Cannibals problem has solved

